

Script de charge et simulation des données

Owner	 louis reynouard
Tags	
Created time	May 1, 2026 5:17 PM

Présentation

Ce script gère les trois cas d'usage (images, texte, tabulaire), les quatre niveaux de charge (nominal, charge, stress, extreme), et produit une sortie structurée exploitable directement dans les livrables.

Le fichier est placé à `scripts/load_test.py` dans le repo de chaque étudiant. En fonction des datasets récupérés il est possible que vous ayez besoin de modifier ce script. documentez tout changement dans un `.md` dédié dans `/script`.

Les données sont placées dans `data/` selon la structure suivante :

```
data/
├── images/                # sous-ensemble HAM10000 (cas 1, fourni
    │                     par l'enseignant)
    │   ├── img_0001.jpg
    │   ├── img_0002.jpg
    │   └── ...
├── comments.csv          # Jigsaw dataset (cas 2, fourni par l'en
seignant)
└── churn.csv             # Telco churn dataset (cas 3, fourni par
l'enseignant)
```

Code complet

```
#!/usr/bin/env python3
"""
load_test.py - Script de charge fourni par l'enseignant.
Ne pas modifier.

Usage :
    python scripts/load_test.py --case images --level nominal
--url http://HOST:PORT/predict
    python scripts/load_test.py --case text --level charge
--url http://HOST:PORT/predict
    python scripts/load_test.py --case churn --level stress
--url http://HOST:PORT/predict
    python scripts/load_test.py --case images --level extreme
--rate 400 --url http://HOST:PORT/predict

Récupérer l'URL avec :
    minikube service inference-svc -n projet-TRIGRAMME --url
"""

import argparse
import concurrent.futures
import csv
import json
import os
import random
import sys
import time
from pathlib import Path

import requests

# -----
# -----
# Configuration des niveaux prédéfinis
# -----
```

```

-----
LEVELS = {
    "nominal": {"rate": 10, "duration": 300},
    "charge": {"rate": 50, "duration": 300},
    "stress": {"rate": 150, "duration": 300},
    "extreme": {"rate": None, "duration": 300}, # rate fourn
i par --rate
}

# -----
# Chemins des données
# -----
DATA_ROOT = Path(__file__).parent.parent / "data"

DATA_PATHS = {
    "images": DATA_ROOT / "images",
    "text": DATA_ROOT / "comments.csv",
    "churn": DATA_ROOT / "churn.csv",
}

# -----
# Fonctions d'envoi par cas d'usage
# -----

def send_image(url: str, image_path: Path) -> tuple[int, floa
t]:
    """
    Envoie une image JPEG en multipart/form-data.
    Endpoint attendu : POST /predict
    Payload : champ 'file' contenant le fichier JPEG.
    Réponse attendue : JSON {"prediction": str, "confidence":
float}

```

```

"""
with open(image_path, "rb") as f:
    files = {"file": (image_path.name, f, "image/jpeg")}
    t0 = time.monotonic()
    resp = requests.post(url, files=files, timeout=30)
    return resp.status_code, time.monotonic() - t0

def send_text(url: str, text: str) -> tuple[int, float]:
    """
    Envoie un commentaire texte en JSON.
    Endpoint attendu : POST /predict
    Payload : {"text": "..."} (UTF-8, Content-Type: application/json)
    Réponse attendue : JSON {"label": str, "score": float}
    """
    payload = {"text": text}
    t0 = time.monotonic()
    resp = requests.post(url, json=payload, timeout=10)
    return resp.status_code, time.monotonic() - t0

def send_churn(url: str, row: dict) -> tuple[int, float]:
    """
    Envoie un profil client en JSON.
    Endpoint attendu : POST /predict
    Payload : dict avec les colonnes du dataset Telco (toutes les colonnes features,
                sans la colonne cible 'Churn').
    Réponse attendue : JSON {"churn_probability": float, "recommended_offer": str}
    """
    t0 = time.monotonic()
    resp = requests.post(url, json=row, timeout=10)
    return resp.status_code, time.monotonic() - t0

```

```

# -----
# Chargement des données
# -----

def load_data(case: str) -> list:
    """Charge le pool de données pour le cas d'usage."""
    if case == "images":
        path = DATA_PATHS["images"]
        if not path.exists():
            sys.exit(f"[ERREUR] Répertoire images introuvable
: {path}")
        files = list(path.glob("*.jpg")) + list(path.glob("*.
jpeg"))
        if not files:
            sys.exit(f"[ERREUR] Aucune image JPEG trouvée dan
s {path}")
        print(f"[INFO] {len(files)} images chargées depuis {p
ath}")
        return files

    elif case == "text":
        path = DATA_PATHS["text"]
        if not path.exists():
            sys.exit(f"[ERREUR] Fichier CSV introuvable : {pa
th}")
        with open(path, encoding="utf-8") as f:
            reader = csv.DictReader(f)
            comments = [row["comment_text"] for row in reader
                        if row.get("comment_text")]
        if not comments:
            sys.exit(f"[ERREUR] Aucun commentaire trouvé dans
comments.csv")
        print(f"[INFO] {len(comments)} commentaires chargés d

```

```

e puis {path}")
    return comments

    elif case == "churn":
        path = DATA_PATHS["churn"]
        if not path.exists():
            sys.exit(f"[ERREUR] Fichier CSV introuvable : {path}")

        # Colonnes à exclure (cible + identifiant)
        EXCLUDE = {"Churn", "customerID"}
        with open(path, encoding="utf-8") as f:
            reader = csv.DictReader(f)
            rows = [{k: v for k, v in row.items() if k not in EXCLUDE}

                    for row in reader]

            if not rows:
                sys.exit("[ERREUR] Aucune ligne trouvée dans churn.csv")

            print(f"[INFO] {len(rows)} profils clients chargés de puis {path}")
            return rows

    else:
        sys.exit(f"[ERREUR] Cas d'usage inconnu : {case}")

# -----
# -----
# Boucle de test
# -----
# -----

def build_task(case: str, data: list, url: str):
    """Retourne une fonction de tâche adaptée au cas d'usage. """
    if case == "images":

```

```

        def task(_):
            return send_image(url, random.choice(data))
    elif case == "text":
        def task(_):
            return send_text(url, random.choice(data))
    elif case == "churn":
        def task(_):
            return send_churn(url, random.choice(data))
    return task

def run_test(case: str, rate: int, duration: int, url: str) -
> dict:
    """
    Exécute le test de charge et retourne les métriques.
    rate      : requêtes par minute
    duration: durée en secondes
    """
    data = load_data(case)
    task = build_task(case, data, url)
    interval = 60.0 / rate
    end_time = time.monotonic() + duration

    futures = []
    statuses = []
    latencies = []

    print(f"[TEST] case={case}  rate={rate} req/min  duration
={duration}s")
    print(f"[TEST] URL : {url}")
    print(f"[TEST] Début : {time.strftime('%H:%M:%S')}")

    with concurrent.futures.ThreadPoolExecutor(max_workers=4
0) as executor:
        while time.monotonic() < end_time:
            futures.append(executor.submit(task, None))

```

```

        time.sleep(interval)

    for f in concurrent.futures.as_completed(futures):
        try:
            status, latency = f.result()
        except requests.exceptions.Timeout:
            status, latency = 408, 30.0
        except Exception as e:
            status, latency = 0, 30.0
        statuses.append(status)
        latencies.append(latency)

n = len(latencies)
n200 = statuses.count(200)
sorted_lat = sorted(latencies)

results = {
    "case": case,
    "level": f"extreme (rate={rate})" if rate not in (10, 50, 150) else {
        10: "nominal", 50: "charge", 150: "stress"
    }[rate],
    "rate_configured": rate,
    "duration_s": duration,
    "total_requests": n,
    "success_200": n200,
    "failure": n - n200,
    "success_rate_pct": round(n200 / n * 100, 1) if n else 0,
    "latency_avg_s": round(sum(latencies) / n, 3) if n else 0,
    "latency_p95_s": round(sorted_lat[max(0, int(0.95 * n) - 1)], 3) if n else 0,
    "latency_max_s": round(sorted_lat[-1], 3) if n else 0,
}

```



```

    }
    return results

def print_results(r: dict) -> None:
    print("\n" + "=" * 55)
    print(f"  RÉSULTATS  -  {r['case'].upper()} / {r['level'].upper()}")
    print("=" * 55)
    print(f"  Rate configuré      : {r['rate_configured']} req/min")
    print(f"  Durée                  : {r['duration_s']}s")
    print(f"  Requêtes envoyées    : {r['total_requests']}")
    print(f"  Succès (HTTP 200)    : {r['success_200']}")
    print(f"  Échecs                : {r['failure']}")
    print(f"  Taux de succès       : {r['success_rate_pct']}%")
    print(f"  Latence moyenne      : {r['latency_avg_s']}s")
    print(f"  Latence P95          : {r['latency_p95_s']}s")
    print(f"  Latence max          : {r['latency_max_s']}s")
    print("=" * 55)

# -----
# -----
# Point d'entrée
# -----
# -----

def parse_args():
    parser = argparse.ArgumentParser(
        description="Script de charge pour le TP Orchestration."
    )
    parser.add_argument(
        "--case",
        choices=["images", "text", "churn"],

```

```

        required=True,
        help="Cas d'usage : images | text | churn"
    )
    parser.add_argument(
        "--level",
        choices=["nominal", "charge", "stress", "extreme"],
        required=True,
        help="Niveau de charge"
    )
    parser.add_argument(
        "--rate",
        type=int,
        default=None,
        help="Requêtes par minute (obligatoire pour --level e
xtreme)"
    )
    parser.add_argument(
        "--url",
        required=True,
        help="URL complète du endpoint /predict du service
d'inférence"
    )
    parser.add_argument(
        "--duration",
        type=int,
        default=300,
        help="Durée du test en secondes (défaut : 300)"
    )
    return parser.parse_args()

def main():
    args = parse_args()

    level_config = LEVELS[args.level]
    rate = args.rate if args.level == "extreme" else level_co

```

```

nfig["rate"]

    if args.level == "extreme" and rate is None:
        sys.exit("[ERREUR] --level extreme requiert --rate N
(ex: --rate 400)")

    results = run_test(
        case=args.case,
        rate=rate,
        duration=args.duration,
        url=args.url,
    )
    print_results(results)

if __name__ == "__main__":
    main()

```

Format des réponses attendues par service

Le script ne contrôle pas le format de réponse au-delà du code HTTP. Il compte `200` comme succès et tout autre code comme échec. Cependant, pour que le service de monitoring puisse enregistrer les prédictions, le service d'inférence doit retourner un JSON structuré.

Cas 1 (images) :

```

{
  "prediction": "malignant",
  "confidence": 0.87,
  "class_detail": "melanoma"
}

```

Cas 2 (texte) :

```
{
  "label": "toxic",
  "score": 0.92,
  "categories": ["insult", "obscene"]
}
```

Cas 3 (churn) :

```
{
  "churn_probability": 0.74,
  "recommended_offer": "remise_tarifaire"
}
```

Toute réponse avec un code HTTP autre que 200, ou une réponse sans corps JSON valide, est comptée comme un échec dans les métriques du service de monitoring.

Comment récupérer l'URL du service

```
# Option 1 : URL directe via minikube service
minikube service inference-svc -n projet-TRIGRAMME --url

# Option 2 : tunnel en arrière-plan (nécessaire si le service
est de type LoadBalancer)
minikube tunnel &
kubectl get svc inference-svc -n projet-TRIGRAMME
# Utiliser EXTERNAL-IP:PORT

# Option 3 : port-forward temporaire
kubectl port-forward svc/inference-svc 8080:80 -n projet-TRIGRAMME &
# URL : http://localhost:8080/predict
```

L'URL utilisée pendant les tests doit être documentée dans le livrable de chaque séance.

Exemples de lancements

```
# Test nominal - cas images
python scripts/load_test.py \
    --case images \
    --level nominal \
    --url http://192.168.49.2:31234/predict

# Test de charge - cas texte
python scripts/load_test.py \
    --case text \
    --level charge \
    --url http://192.168.49.2:31234/predict

# Test stress - cas churn
python scripts/load_test.py \
    --case churn \
    --level stress \
    --url http://192.168.49.2:31234/predict

# Stress test extrême - cas images, 400 req/min
python scripts/load_test.py \
    --case images \
    --level extreme \
    --rate 400 \
    --url http://192.168.49.2:31234/predict

# Stress test extrême - durée réduite pour un test rapide
python scripts/load_test.py \
    --case images \
    --level extreme \
```

```
--rate 200 \  
--duration 60 \  
--url http://192.168.49.2:31234/predict
```

Dépendances

```
requests>=2.31.0
```

Installation :

```
pip install requests
```

Le script utilise uniquement la bibliothèque standard Python et `requests`. Aucune autre dépendance n'est requise.